DOCUMENTO UBUNTU SERVER

Nacho Abad Rodríguez

Índice

1. Introducción 2
2. Objetivos 3
3. Contenidos 4
3.1 Instalación Zabbix 4
3.1.1 Configuración Terminal 4
3.1.2 Configuración Interfaz Web 8
3.2 IPTables + VPN WireGuard 9
3.2.1 Firewall IPTables 9
3.2.2 VPN WireGuard 11
TERMINAR EN UN ENTORNO SEGURO 15
3.3 Instalación GLPI + Conectividad Zabbix 16
3.4 Conectividad Zabbix-GLPI y pruebas 21
4. Conclusión 28
5. Bibliografía 29

1. Introducción

El propósito de este documento es describir el despliegue, configuración y puesta en funcionamiento de un entorno orientado a la monitorización y a la gestión de activos utilizando tecnologías ampliamente adoptadas en la administración de sistemas: Zabbix, GLPI, IPTables y WireGuard. Todo el proceso se realiza sobre un entorno controlado de laboratorio basado en máquinas virtuales, con el objetivo de reproducir escenarios reales sin comprometer sistemas en producción.

A lo largo del documento se detalla la instalación de cada componente, su configuración interna, la integración entre las distintas herramientas y la verificación operativa del entorno resultante. Aunque las credenciales y configuraciones utilizadas son válidas funcionalmente, se han simplificado con fines didácticos y no representan prácticas recomendadas para sistemas críticos o entornos empresariales.

El conjunto proporciona una visión de monitorizar, notificar, registrar y gestionar incidencias de manera automatizada. Esto permite comprender cómo interactúan las diferentes capas de un sistema moderno de supervisión y soporte técnico, desde el servidor de monitorización hasta la apertura automática de tickets.

2. Objetivos

- Instalar y configurar Zabbix para la monitorización del sistema.
- Configurar IPTables y WireGuard para mejorar la seguridad y el acceso remoto.
- Instalar GLPI como plataforma de gestión de activos y tickets.
- Conectar Zabbix con GLPI para la creación automática de incidencias.
- Verificar el funcionamiento conjunto de todas las herramientas.
- Aprender y pasarlo bien con la tecnología

3. Contenidos

Vamos a partir de una base de una máquina virtual Ubuntu Server v24.0.4.4 con 8GB de RAM estables y en adaptador puente.

3.1 Instalación Zabbix

3.1.1 Configuración Terminal

Lo primero que vamos a hacer va a ser actualizar los paquetes y después instalar el repositorio oficial de Zabbix

```
sudo apt update && sudo apt upgrade -y
```

```
wget https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/zabbix-release/zabbix-release_7.0-2+ubuntu24.04_all.deb
```

```
sudo dpkg -i zabbix-release_7.0-2+ubuntu24.04_all.deb
```

```
sudo apt update
```

Instalamos el Zabbix server, frontend, su agente y dependencias:

```
sudo apt install zabbix-server-mysql zabbix-frontend-php zabbix-apache-conf zabbix-sql-scripts zabbix-agent -y
```

```
sudo apt install mysql-server -y
```

Ahora creamos la base de datos con el usuario zbxZero y su contraseña.

```
sudo mysql -u root -p
CREATE DATABASE zabbix CHARACTER SET utf8mb4 COLLATE utf8mb4_bin;
CREATE USER 'zbxZero'@'localhost' IDENTIFIED BY 'Cliente.1';
GRANT ALL PRIVILEGES ON zabbix.* TO 'zbxZero'@'localhost';
SET GLOBAL log_bin_trust_function_creators = 1;
FLUSH PRIVILEGES;
EXIT;
```

Ahora importamos todo el esquema de la base de datos desglosado para que Zabbix pueda leerlos mejor. Es normal que tarde un rato, pero no hay problema.

```
zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz | mysql --default-character-set=utf8mb4 -uzbxZero -p zabbix
```

Desactivamos el permiso temporal establecido previamente en la base de datos:

```
sudo mysql -u root -p
SET GLOBAL log_bin_trust_function_creators = 0;
EXIT;
```

Editamos la configuración desde el archivo etc/zabbix/zabbix_server.conf:

```
# Mandatory: yes
# Default:
# DBName

DBName=zabbix

### Option: DBSchema
#       Schema name. Used for PostgreSQL.
#
# Mandatory: no
# Default:
# DBSchema=

### Option: DBUser
#       Database user.
#
# Mandatory: no
# Default:
# DBUser=

DBUser=zbxZero

### Option: DBPassword
#       Database password.
#       Comment this line if no password is used.
#
# Mandatory: no
# Default:
# DBPassword=
DBPassword=Cliente.1

### Option: DBSocket
#       Path to MySQL socket.
#
# Mandatory: no
```

Desde el archivo /etc/php/8.3/apache2/php.ini configuramos la zona horaria del frontend.

Des comentamos la línea ;date.timezone = y la dejamos en date.timezone = Europe/Madrid

Reiniciamos el sistema de apache y arrancamos todo:

sudo systemctl restart apache2

sudo systemctl restart zabbix-server zabbix-agent apache2

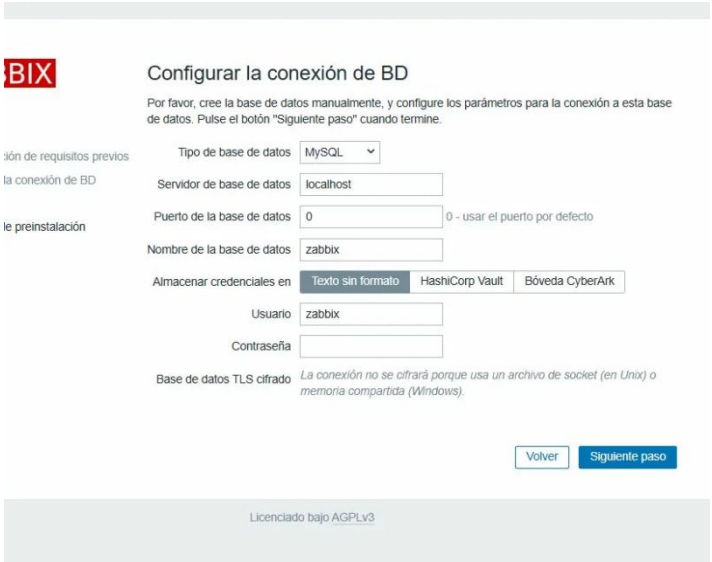
sudo systemctl enable zabbix-server zabbix-agent apache2

Ahora ya Podemos entrar a Zabbix desde el navegador en otro dispositivo con la url “http://IP_DE_TU_SERVIDOR/zabbix” y esta será la bienvenida de Zabbix:

3.1.2 Configuración Interfaz Web

Desde el navegador accedemos a http://IP_DE_LA_MÁQUINA/zabbix y nos aparece la bienvenida en la que le damos a siguiente paso:

Nos pedirá una serie de parámetros para configurar la base de datos. Ponemos los datos necesarios y tras una serie de pantallas sencillas, habremos logrado tener el Zabbix en completo funcionamiento.



3.2 IPTables + VPN WireGuard

Esta parte es totalmente opcional ya que no interviene directamente como ayuda para la conectividad entre Zabbix y GLPI. Dicho esto, vamos a proseguir con la configuración de reglas de firewall IPTables e instalación de un VPN.

3.2.1 Firewall IPTables

La idea de este Firewall va a ser prohibir todas las conexiones origen hacia este equipo menos las permitidas como por ssh, al Zabbix, o en un futuro al GLPI.

Lo primero va a ser instalar la herramienta IPTables y también IPTables-persistent para ayudarnos a guardar las reglas, aunque apaguemos el equipo (qué no deberíamos).

```
sudo apt install iptables -y

sudo apt install iptables-persistent -y
```

Ya teniendo instalado IPTables vamos a hacer una limpieza de tablas en caso de que ya lo tuviéramos instalado y posteriormente comenzamos con políticas por defecto:

```
sudo iptables -F

sudo iptables -X

sudo iptables -P INPUT   DROP    # Bloquear todo lo entrante

sudo iptables -P FORWARD DROP    # No somos router

sudo iptables -P OUTPUT ACCEPT # Salida libre

sudo iptables -A INPUT -i lo -j ACCEPT #Loopback
```

Aquí permitimos las comunicaciones en las que el origen somos nosotros.

sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

sudo iptables -A INPUT -m conntrack --ctstate INVALID -j DROP

Estas reglas las vamos a utilizar para permitir las comunicaciones necesarias:

sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT #SSH

sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT #HTTP

sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT #HTTPS

sudo iptables -A INPUT -p tcp --dport 10051 -j ACCEPT #AgentesZabbix

sudo iptables -A INPUT -p udp --dport 51820 -j ACCEPT #VPN WireGuard

Guardamos de forma persistente las reglas con el siguiente comando

sudo netfilter-persistent save

Y con el siguiente comando visualizamos todas las tablas y reglas creadas:

sudo iptables -L INPUT -v -n --line-numbers

```
zero@servernach:~$ sudo iptables -L INPUT -v -n --line-numbers
Chain INPUT (policy DROP 9826 packets, 1309K bytes)
num   pkts bytes target     prot opt in     out     source               destination
1    89166 8021K ACCEPT     0    --  lo     *       0.0.0.0/0            0.0.0.0/0
2    58962   45M ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate RELATED,ESTABLISHED
3        1    52 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:22
4     1886 98072 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:80
5        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:443
6        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:10051
7        0     0 ACCEPT     17   --  *      *       0.0.0.0/0            0.0.0.0/0            udp dpt:51820
8        0     0 ACCEPT     17   --  *      *       0.0.0.0/0            0.0.0.0/0            udp dpt:51820
zero@servernach:~$
```

3.2.2 VPN WireGuard

Este VPN lo hemos instalado y configurado simplemente para comprobar formas de conectarnos a la máquina desde otros dispositivos y redes. Puede ser muy básico, pero es la primera vez que pruebo este tipo de cosas. Cualquier comentario de ayuda o mejora será recibido con agradecimiento

Instalamos wireguard y generamos un par de claves del servidor para wireguard:

sudo apt install wireguard

wg genkey | sudo tee /etc/wireguard/privatekey | wg pubkey | sudo tee /etc/wireguard/publickey

sudo chmod 600 /etc/wireguard/privatekey

Ahora en el archivo /etc/wireguard/wg0.conf ponemos el siguiente contenido:

[Interface]

PrivateKey = <contenido de /etc/wireguard/privatekey>

Address = 10.0.0.1/24

ListenPort = 51820

La IP utilizada será la dirección del servidor dentro del túnel VPN

Abrimos el puerto en IPTables para poder utilizarlos si no lo hemos hecho ya y activamos el forwarding:

sudo iptables -A INPUT -p udp --dport 51820 -j ACCEPT

sudo netfilter-persistent save

echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf

sudo sysctl -p

Ahora pasamos a descargar la app WireGuard desde el móvil; el móvil va a ser el dispositivo que utilicemos para conectarnos vía túnel VPN.

Para realizar la conexión de manera mucho más simple vamos a generar un QR desde el servidor, por lo que instalaremos otra herramienta.

```
sudo apt install qrencode
```

También generamos las claves que va a utilizar el móvil en WireGuard:

```
wg genkey | sudo tee /etc/wireguard/mobile_privatekey | wg pubkey | sudo tee /etc/wireguard/mobile_publickey
```

```
sudo chmod 600 /etc/wireguard/mobile_privatekey
```

Creamos el archivo de configuración del móvil “/etc/wireguard/mobile.conf” que contendrá la siguiente información y datos.

[Interface]

PrivateKey = <contenido de /etc/wireguard/mobile_privatekey>

Address = 10.0.0.2/24

DNS = 1.1.1.1

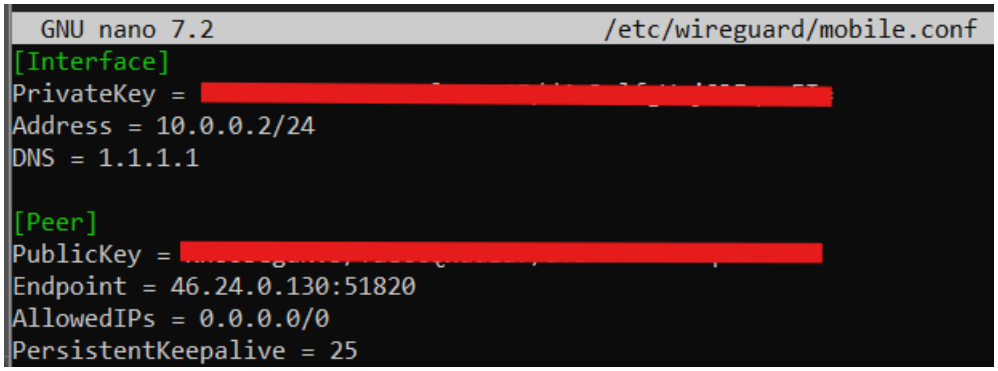
[Peer]

PublicKey = <contenido de /etc/wireguard/publickey>

Endpoint = <ip pública de tu servidor>:51820

AllowedIPs = 0.0.0.0/0

PersistentKeepalive = 25



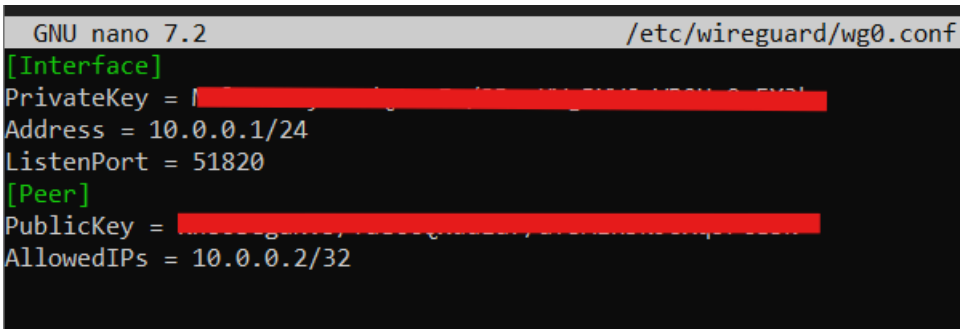
curl ifconfig.me para revisar la ip pública del servidor

Ahora en el archivo de configuración del servidor en el que hemos entrado previamente, añadimos el móvil

[Peer]

PublicKey = <contenido de /etc/wireguard/mobile_publickey>

AllowedIPs = 10.0.0.2/32



Vamos a proceder con la generación del qr con el siguiente comando:

sudo qrencode -t ansiutf8 -r /etc/wireguard/mobile.conf

Teniendo ya el QR, lo escaneamos desde la aplicación del móvil y después reiniciamos WireGuard desde el servidor para aplicar la configuración con el móvil

```
sudo wg-quick down wg0 && sudo wg-quick up wg0
```

Activamos el túnel en el móvil desde la app y probamos a hacernos ping

TERMINAR EN UN ENTORNO SEGURO

3.3 Instalación GLPI + Conectividad Zabbix

Como partimos de una máquina que ya tiene instalado Apache y MySQL será más sencillo todo. Antes de nada, vamos a instalar las extensiones de PHP que GLPI necesita:

```
sudo apt install php-xml php-curl php-gd php-mbstring php-zip php-bz2 php-intl php-ldap php-mysql php-opcache -y
```

Y descargamos GLPI en el directorio temporal o donde prefiramos

```
wget https://github.com/glpi-project/glpi/releases/download/10.0.17/glpi-10.0.17.tgz
```

Lo descomprimimos, movemos al servidor web y le damos permisos. Después reiniciamos Apache:

```
tar -xzf glpi-10.0.17.tgz
sudo mv glpi /var/www/html/
sudo chown -R www-data:www-data /var/www/html/glpi
sudo chmod -R 755 /var/www/html/glpi
sudo systemctl restart apache2
```

Creamos la base de datos de GLPI:

```
sudo mysql -u root -p

CREATE DATABASE glpi CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'glpiZero'@'localhost' IDENTIFIED BY 'Cliente.1';

GRANT ALL PRIVILEGES ON glpi.* TO 'glpiUser'@'localhost';

FLUSH PRIVILEGES;

EXIT;
```

Pasamos a configurar Apache para GLPI. En el archivo /etc/apache2/sites-available/glpi.conf ponemos el siguiente contenido:

```
<VirtualHost *:80>
    ServerName IP_SERVIDOR
    DocumentRoot /var/www/html/glpi/public

    <Directory /var/www/html/glpi/public>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/glpi_error.log
    CustomLog ${APACHE_LOG_DIR}/glpi_access.log combined
</VirtualHost>
```

Después ejecutamos los siguientes comandos antes de crear el archivo .htaccess

```
sudo a2ensite glpi.conf

sudo a2dissite 000-default.conf

sudo a2enmod rewrite
```

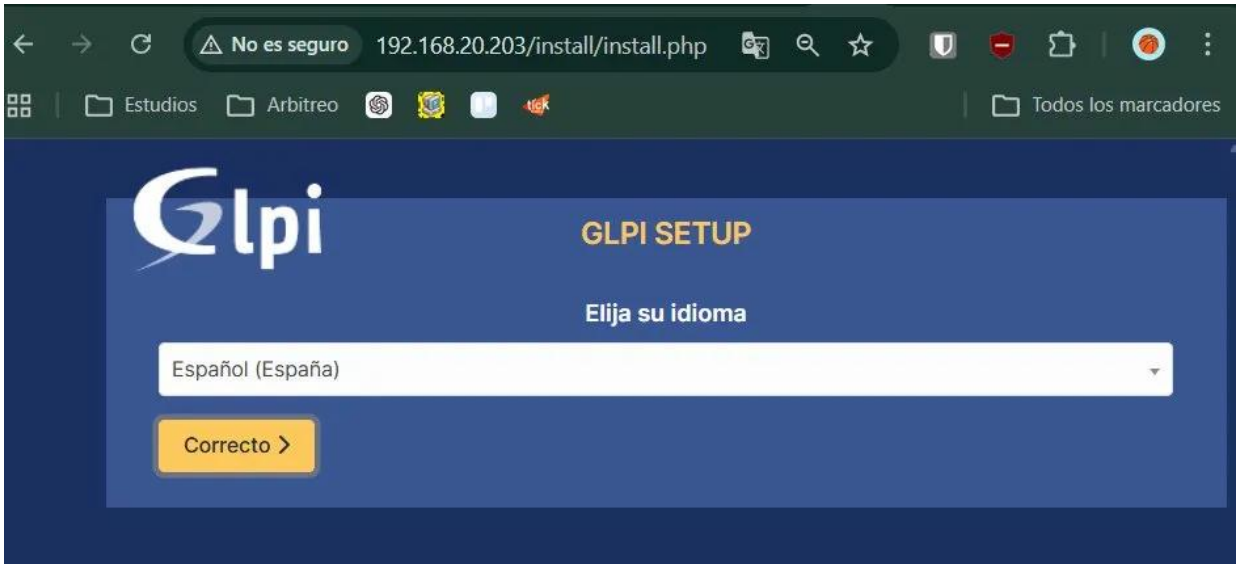
Creamos el archivo /var/www/html/glpi/public/.htaccess y ponemos el siguiente contenido:

```
RewriteEngine On

RewriteCond %{REQUEST_FILENAME} !-f

RewriteRule ^(.*)$ index.php [QSA,L]
```

Cambiamos el propietario y reiniciamos Apache

```
sudo chown www-data:www-data /var/www/html/glpi/public/.htaccess

sudo systemctl restart apache2
```

Ahora ya deberíamos poder acceder desde el navegador en http://IP_DE_LA_MÁQUINA



Después de darle a correcto seguiremos el asistente e introduciremos los datos que sean necesarios según los hemos ido configurando y creando. Puede que en una revisión de seguridad nos dé Warnings, pero no hay problema y podremos continuar.

Por algún casual se nos ha escapado darle permisos al usuario glpiZero sobre la base de datos, por lo que se los vamos a dar desde la línea de comandos:

```
sudo mysql -u root -p

GRANT ALL PRIVILEGES ON glpi.* TO 'glpiZero'@'localhost';

FLUSH PRIVILEGES;

EXIT;
```

Después de haber corregido ese error se iniciará la base de datos que suele tardar un par de minutos. Posterior a eso nos dirá que hemos completado la instalación:

Ahora ya tenemos en la misma máquina:

- Zabbix en http://192.168.20.203/zabbix — monitorización
- GLPI en http://192.168.20.203 — gestión de activos y helpdesk

3.4 Conectividad Zabbix-GLPI y pruebas

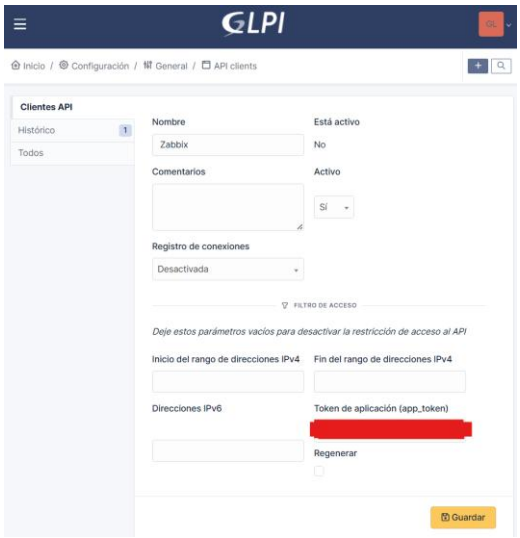
Este paso es el más importante y por el que hemos realizado el resto de ellos. Queremos crear una comunicación entre Zabbix y GLPI. Por ejemplo, cuando Zabbix detecta un problema en un servidor, puede abrir automáticamente un ticket en GLPI. Así tienes:

- Zabbix detectando y alertando
- GLPI gestionando y registrando la resolución

[Ejemplo práctico] Imagina que tienes 50 ordenadores en una empresa:

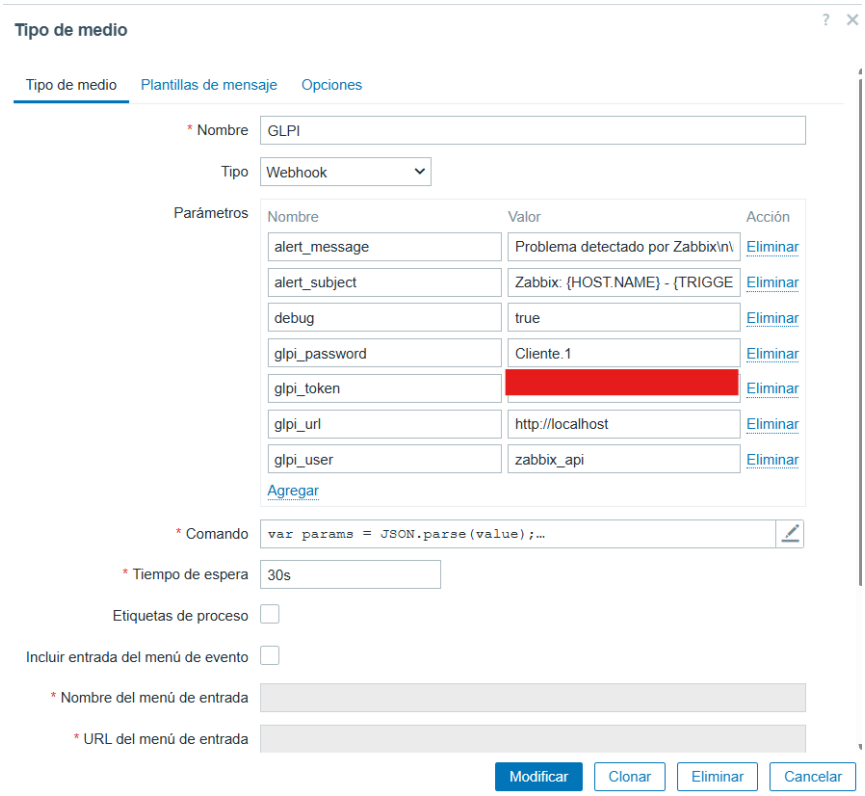
1. GLPI sabe que el ordenador PC-042 lo usa Juan del departamento de contabilidad, tiene Windows 11, 16GB RAM, garantía hasta 2026 y licencia de Office
2. Zabbix monitoriza ese equipo y detecta que el disco está al 95%
3. Zabbix abre automáticamente un ticket en GLPI
4. El técnico recibe la alerta, amplía el disco y cierra el ticket
5. Queda todo registrado en el historial del equipo

Primeramente, deberemos activar una API en GLPI para posteriormente crear un usuario con esta. En la configuración de API de GLPI añadiremos un cliente API con los siguientes parámetros:

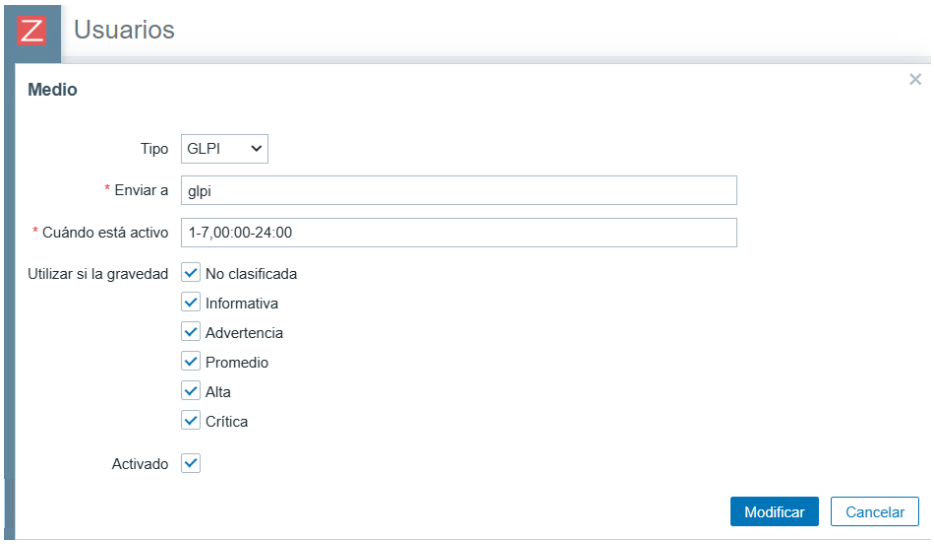


Teniendo activada esta API, crearemos un usuario también en GLPI con el nombre zabbix_api y con perfil de admin.

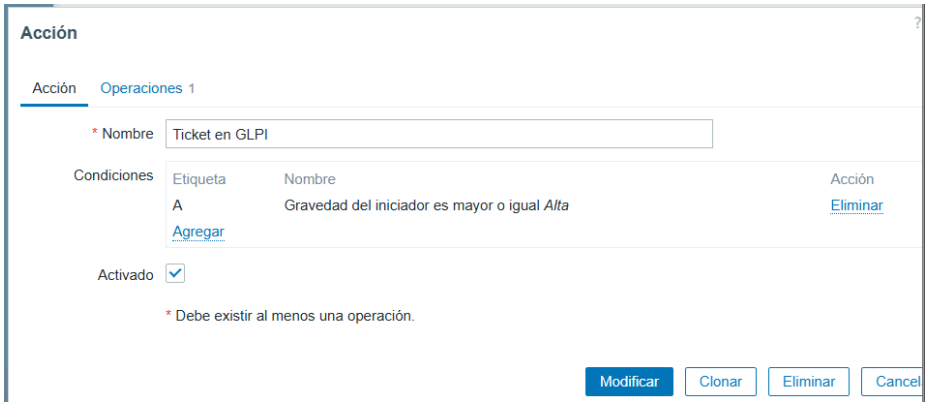
Desde Zabbix vamos a crear un tipo de medio para GLPI con los siguientes parámetros, que podremos encontrar en el repositorio de GitHub.

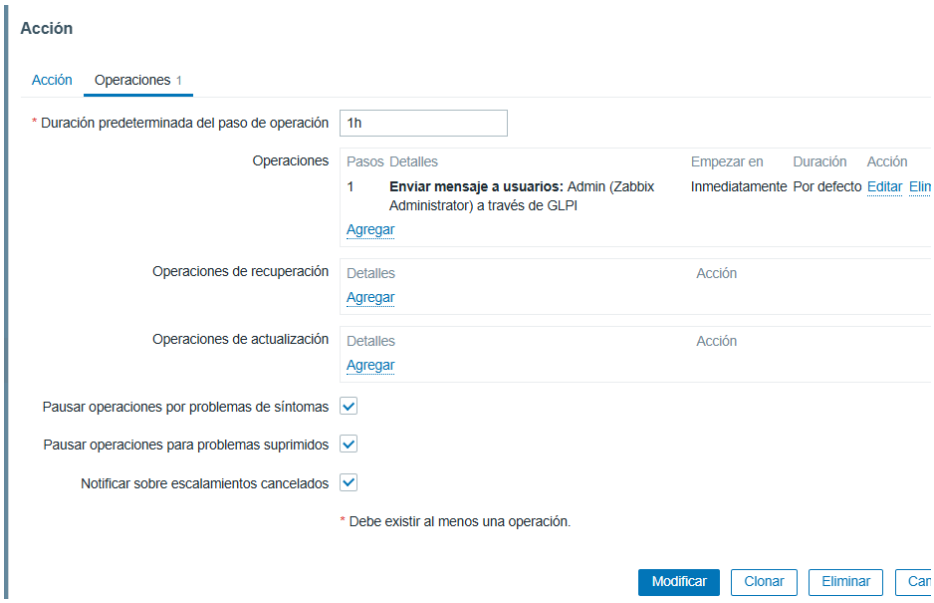


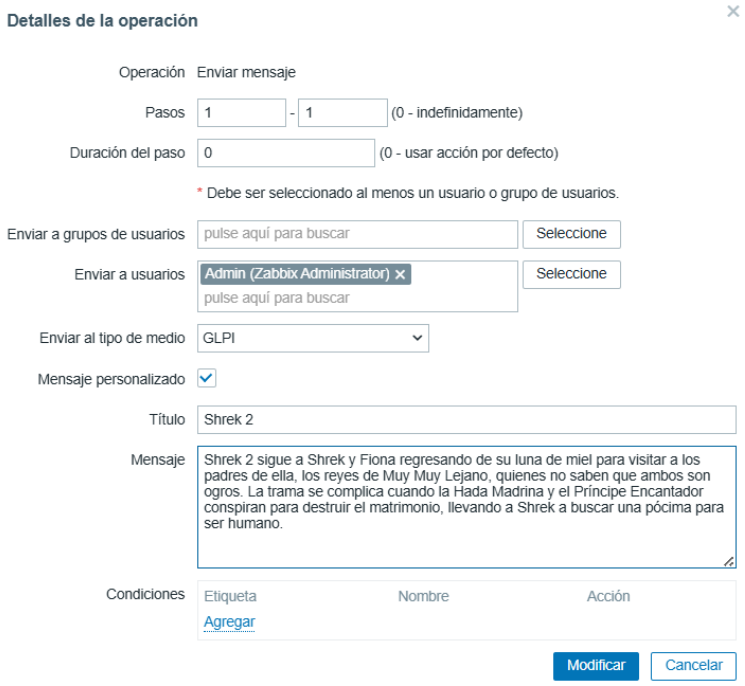
Cuando se modifique/cree este medio, una buena práctica será darle a probar este medio y comprobar que no nos dé ningún error. Teniendo ya el medio creado vamos a asignarlo al usuario Admin con todas las severidades marcadas.



Ahora creamos la acción de crear el ticket para GLPI. Desde alertas crearemos la acción con los siguientes parámetros:







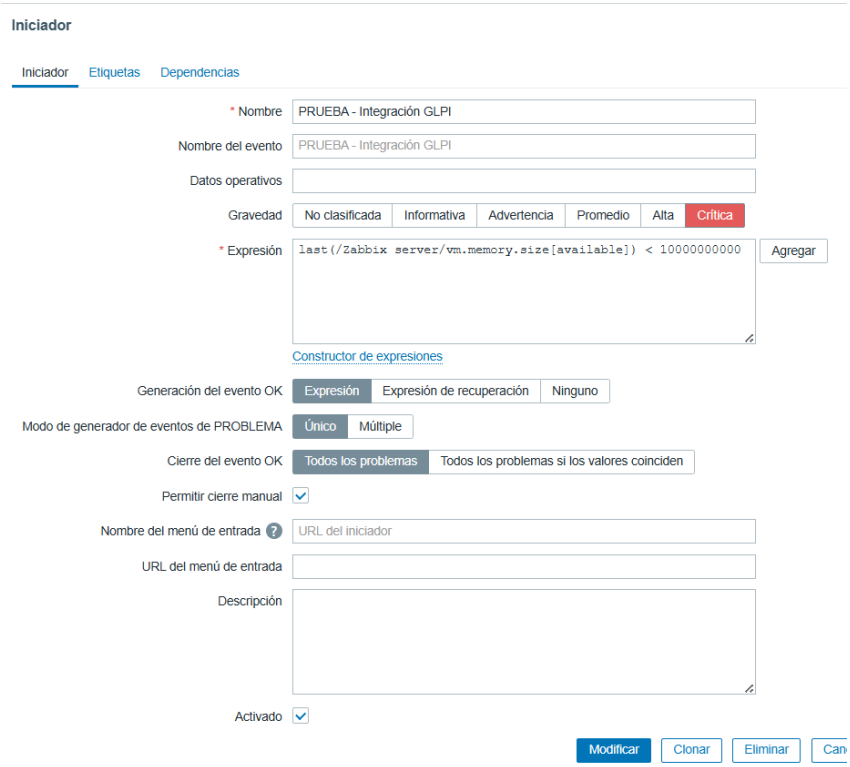
Un error común suele ser no activar el mensaje personalizado o no rellenarlo. Nosotros hemos puesto un contenido chorra, pero estos son parámetros que sí son útiles:

Zabbix: {HOST.NAME} - {TRIGGER.NAME}

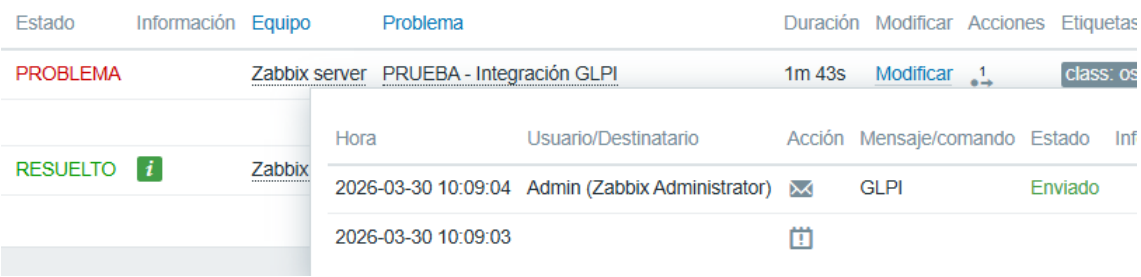
Problema detectado: {TRIGGER.NAME}\nHost: {HOST.NAME}\nSeveridad: {TRIGGER.SEVERITY}

Teniendo la conectividad completada, vamos a realizar una prueba sencilla. Esta prueba consistirá en una alerta que se active fácilmente y por consecuencia cree un ticket que deberemos observar desde GLPI, si todo funciona correctamente.

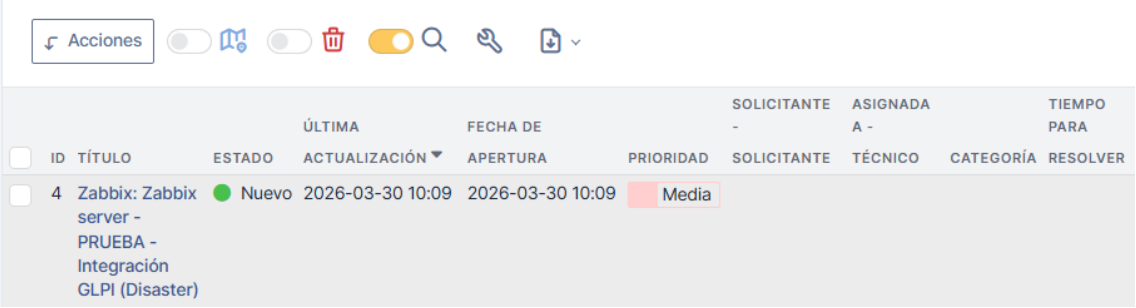
Seleccionamos los iniciadores/triggers del host Zabbix Server y creamos uno.



Teniendo ya la alerta creada deberíamos pasar a ver como se activa y vemos un problema desde la monitorización:



Alrededor de 30-60s veremos como el ticket se ha creado en GLPI:



4. Conclusión

La construcción de este entorno ha permitido integrar de forma práctica varias herramientas clave en la administración de sistemas, demostrando cómo pueden complementarse para ofrecer una gestión centralizada, automatizada y eficiente. La instalación y configuración de Zabbix proporciona una plataforma robusta de monitorización capaz de detectar incidencias en tiempo real, mientras que GLPI aporta la capa necesaria para la gestión de activos y la trazabilidad operativa a través de su sistema de tickets.

La incorporación de un firewall basado en IPTables y un túnel seguro mediante WireGuard añade una capa adicional de control y seguridad, reforzando las buenas prácticas que deben acompañar cualquier infraestructura, incluso en entornos de laboratorio.

Finalmente, la integración entre Zabbix y GLPI demuestra el valor de automatizar el flujo de trabajo: cuando un evento crítico se produce, el sistema es capaz de registrarlo, generar un ticket y asociarlo al inventario correspondiente sin intervención manual, ofreciendo un modelo escalable y aplicable a entornos reales de soporte técnico y operación.

Este proyecto, aun desarrollado en un entorno de pruebas, establece una base sólida para implementar soluciones más complejas o llevar estas mismas herramientas a un despliegue de producción siguiendo políticas de seguridad y mantenimiento adecuadas.

5. Bibliografía

https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/zabbix-release/zabbix-release_7.0-2+ubuntu24.04_all.deb

https://glpi-install.readthedocs.io/en/latest/index.html#

https://github.com/glpi-project/glpi/releases/download/10.0.17/glpi-10.0.17.tgz

https://www.youtube.com/watch?v=NMBEVn2ysks

https://github.com/farnacho/Ecosistema-Windows-Server-Zabbix-GLPI